

Sprint 1 Planning & Database Design

Established the implementation baseline for the Cardiac Patient Monitoring API and separated current schema from future clinical-platform scope.

MAIN OBJECTIVE

Document the real EF Core database model.

SPRINT GOAL

Complete, documented schema for patients, vital signs, medications, appointments, and users.

WORK COMPLETED

Schema audit · normalization review · DBML/ERD · risk log

Inspect → model → document

Database Design Baseline

The current ERD documents implemented tables—not the larger planned clinical-platform model.

APPLICATION ENTITIES

**Patients · VitalSigns · Medications ·
Appointments**

INFRASTRUCTURE

**ASP.NET Core Identity · EF
Migrations History**

RELATIONS

**Patient –
Patient –
Patient –
Day 1
Patient <**

ERD



Design Output & Recorded Decisions

DELIVERED

- Sprint goal and backlog
- Normalization review
- DBML + ERD asset
- Schema risk register

DATABASE EVIDENCE

- InitialCreate migration
- Migration designer + model snapshot
- Seeded clinical data

FUTURE SCOPE IDENTIFIED

- Clinician relationship
- Devices and readings
- Alerts, care plans, audit log
- Patient-user linkage

Recorded risks: free-text clinician; role services not registered; InMemory does not prove SQL Server behavior; UTC consistency required.

EF Core Model & Migrations

Converted design decisions into an incremental EF Core migration, a seeded reference table, and an intentional clinical-history delete rule.

MAIN OBJECTIVE

Make the data model explicit and migration-backed.

NEW MODEL ELEMENT

CareCategory
Reference lookup with three seeded risk categories.

KEY DECISION

Appointments changed from cascade delete to restrict.

Model, Relationships & Seed Data

FLUENT API CONFIGURATION

Required/max-length fields
Vital-sign decimal precision
Patient → VitalSign / Medication:
cascade
Patient → Appointment: restrict

CARE CATEGORIES SEED

Low Risk
Moderate Risk
High Risk
Required Name limited to 80
characters

MIGRATION

20260824170100_Day2EntitySetup
Creates CareCategories
Inserts 3 rows
Changes appointment FK to Restrict

Database Change Summary

IMPLEMENTED

CareCategory entity, DbSet, HasData, migration
Appointment FK changed to Restrict

NOT VERIFIED

Live SQL Server migration application
Database GUI verification

NOT IMPLEMENTED

No CareCategory controller, service, DTO, or endpoint

Core Routes I: Patient Catalog

Replaced an unpaginated patient list with an authenticated catalog endpoint designed for controlled browsing.

ENDPOINT

GET /api/patients
JWT protected

QUERY CAPABILITIES

Pagination · search · gender
birth-date filter · four sort modes

RESPONSE

PagedResponseDto<PatientResponseDto>

Controlled Patient Browsing

REQUEST CONTRACT

page (default 1) · pageSize (1–100)
search · gender · bornBefore
sort: nameAsc, nameDesc, oldest,
newest

QUERY PIPELINE

AsNoTracking read query
Optional database-side filters
Stable Id tie-breaker
Count before Skip / Take
Direct DTO projection

RESPONSE

items · page · pageSize
totalCount · totalPages
Invalid paging/sort → HTTP 400

Tests, Postman & Result

TESTS

Controller forwards catalog query
JWT catalog request expects 200
Anonymous request expects 401

DOCUMENTED POSTMAN CASES

Pagination
Gender + descending name
Search + date + oldest
Invalid page values

FINAL RESULT

Catalog implementation is present with
DTO projection and authorization.

Verification limit: Day 5 records both catalog HTTP tests were blocked by Windows Event Log permission failure. No executed Postman result was found.

Core Routes II: Order Creation

Added a write workflow that validates stock, calculates server-owned monetary totals, and persists order changes transactionally for relational providers.

ENDPOINT

POST /api/orders
JWT protected

NEW ENTITIES

Product · Order · OrderItem

BUSINESS RULE

Client sends product IDs + quantities,
never price or totals.

Order Processing Flow

1. VALIDATE

Non-empty items; existing patient/customer; positive quantities

2. CHECK

Aggregate duplicate items; verify all products and available stock

3. CALCULATE

**Line totals from stored
UnitPrice; sum
OrderTotal**

4. PERSIST

Decrement stock, create order/items, save and commit if relational

Relationships: Patient → Orders (restrict) · Order → OrderItems (cascade) · Product → OrderItems (restrict)

Money: Product.UnitPrice · OrderItem.LineTotal · Order.OrderTotal — decimal(18,2)

Migrations, Tests & Review Status

IMPLEMENTED

AddOrdersAndProducts migration
AddOrderMonetaryFields migration
Seed product: Remote ECG Monitor

VERIFIED

3 OrderServiceTests passed
Available stock · insufficient stock
Duplicate quantity aggregation

NOT VERIFIED

Order HTTP endpoint
SQL Server transaction run
Pull request / mentor review

Sprint Review & Evidence-Based Close-Out

Classified implementation, test results, and unverified work without treating documentation or unexecuted requests as proof.

CLOSE-OUT FOCUS

Verify what can be proven.

CODE CHANGES

Documentation-only compared with Day 4.

FINAL STATUS

Implemented and partially verified.

Completed Features & Evidence

IMPLEMENTED

Four migrations
JWT-protected API
Paginated patient catalog
Products, orders, order items
Stock validation + server totals

ACTUALLY VERIFIED

3 / 3 focused order tests passed
32 / 34 full-suite tests passed

OPEN

Catalog HTTP verification
Order HTTP/manual verification
SQL Server migration / rollback
Formal acceptance + review

Build recorded: 0 errors; 4 NU1900 vulnerability-data warnings. Two catalog integration tests were blocked by Event Log permission failure.

Sprint 2 Backlog & Retrospective

PRIORITY BACKLOG

Fix test-host logging
Add order endpoint integration tests
Execute catalog/order Postman requests
Verify migrations and rollback on SQL Server
Create acceptance checklist

WHAT WENT WELL

Clear model/migration progression
Server-owned pricing
Focused order business-rule tests
Evidence-based close-out

KEY IMPROVEMENT

Make the test environment independent of Windows Event Log before treating HTTP behavior as verified.

Sprint 1 outcome: implementation present; verification intentionally incomplete.